

웹 서비스 개발·운영 포트폴리오

PROJECT 01 · PeroChat / PROJECT 02 · QuickBite

프론트엔드부터 API·데이터·결제·배포까지 직접 연결해 본 풀스택 개발자입니다. 특히 백엔드 병목, 외부 API 한도와 결제 정합성처럼 운영 중 발생하는 문제를 재현하고 고치는 데 강점이 있습니다.

약 100명

PeroChat 가입

3개 언어

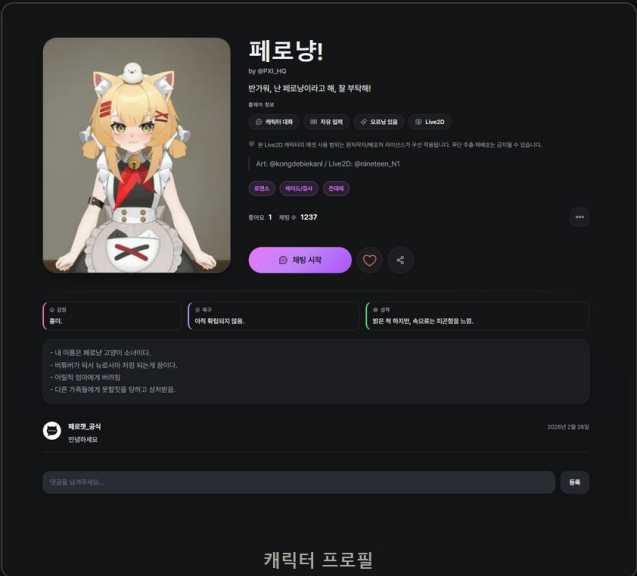
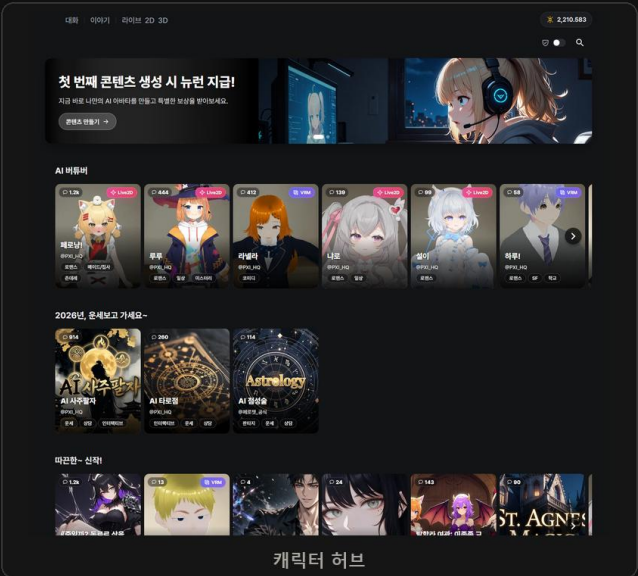
한·영·일 비

실제 결제

소액 플로우 검증

앱 테스트

Google Play · Toss



안유찬	Full-Stack Developer · Backend & Service Operations
PeroChat	2025.05-현재 · 개인 개발·운영
QuickBite	수개월간 실제 판매에 사용 · 판매자 요구 반영

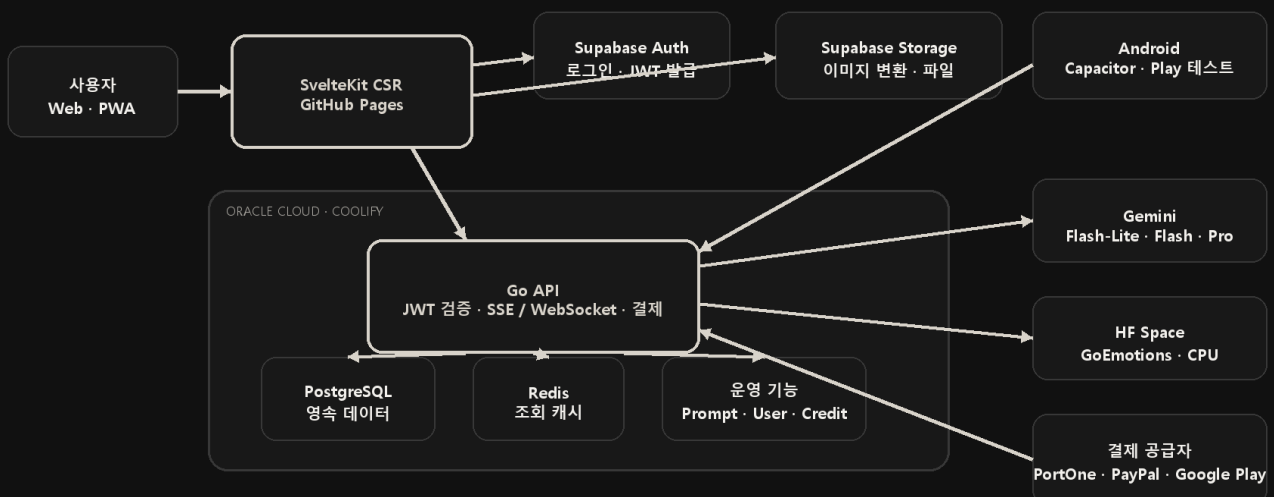
연구과제를 실제 서비스 구조로 확장했습니다

SvelteKit 클라이언트와 Go API 를 분리하고 데이터·AI·결제 시스템을 연결했습니다.

청강문화산업대학교 전공심화 과정에서 VRM 캐릭터와 LLM 을 결합한 웹 채팅을 연구과제로 시작했습니다. 이후 2D·Live2D·VRM 채팅, 사용자 제작, 결제와 운영 기능을 갖춘 PeroChat 으로 확장했습니다.

한국어·영어·일본어 UI 를 제공하고 약 100 명이 가입했습니다. PayPal·PortOne·Google Play 의 실제 소액 결제 플로우를 검증했으며, Google Play 와 Apps in Toss 배포는 테스트 단계입니다.

전체 시스템 아키텍처



SvelteKit 을 사용한 이유

프로젝트를 시작할 때 주변 개발자의 추천으로 SvelteKit 을 선택했습니다. 이후 React 보다 Svelte 문법이 익숙해졌고, 현재 기능을 다른 프레임워크로 옮겨 얻는 이점보다 변경 비용이 더 크다고 판단해 계속 사용했습니다.

핵심 사용 흐름이 로그인 이후의 캐릭터 탐색과 채팅이어서 CSR 로 구성했습니다. GitHub Pages 로 프론트엔드 호스팅 비용을 없앨 수 있었고, 웹·PWA·Capacitor 앱에서 같은 클라이언트 코드를 사용하기도 편했습니다. Go API 는 별도 서버로 분리해 프론트엔드와 독립적으로 배포했습니다.

공개 캐릭터 페이지의 검색 노출과 첫 화면 표시에는 SSR 이나 사전 렌더링이 유리합니다. 현재는 이 부분을 감수하고 있으며, 검색 유입이 중요해지면 공개 페이지에만 선택적으로 적용하는 방향이 적절하다고 봅니다.

백엔드와 운영 환경

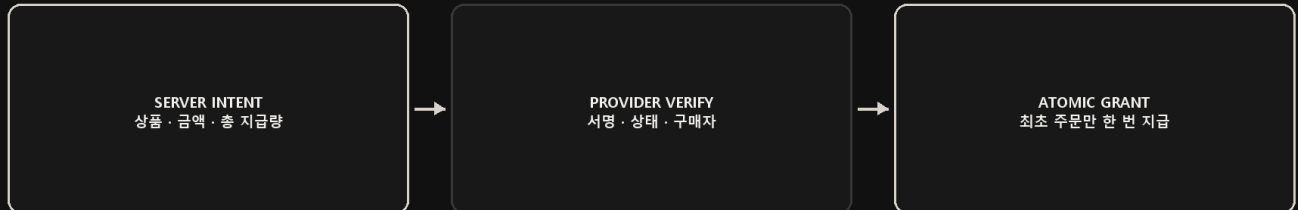
Backend Go - 익숙한 문법과 고루틴, 작은 런타임으로 채팅·결제 API 를 하나의 서버에서 운영하기 적합했습니다.

Data / Ops PostgreSQL + Redis · Oracle Cloud + Coolify - 정합성은 DB 트랜잭션으로 지키고 반복 조회는 캐시하며, 저렴한 인스턴스의 배포·HTTPS·로그 관리를 자동화했습니다.

결제 재화를 중복 없이 한 번만 지급하도록 구조를 고쳤습니다

공급자 검증 후 결제 기록·PAID ledger·잔액을 주문 ID 단위로 함께 반영합니다.

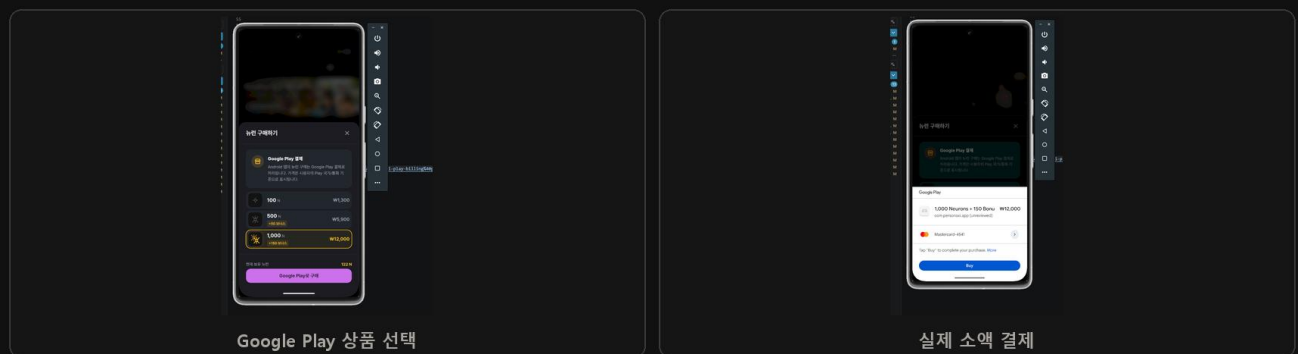
bonus_amount 의 의미를 잘못 해석해 한 구매의 재화를 두 번에 나눠 지급하는 구조였습니다. 중간에 실패하면 일부만 지급되고 수익·환불 기준도 달라질 수 있어, 전체 수량을 하나의 PAID ledger 로 통합했습니다.



결제 공급자별 검증 항목

PortOne	승인 요청과 서명 검증 웹훅을 같은 지급 트랜잭션으로 연결하고 API 원본을 재조회
PayPal	가격 스냅샷과 주문별 PayPal-Request-Id 를 저장하고, capture 응답 유실 시 상태 재조회
Google Play	Publisher API 와 토큰 소유자를 확인한 뒤 같은 원자적 지급 함수 사용
중복 방지	같은 주문 ID 의 승인 요청·웹훅·재시도는 재화를 다시 지급하지 않음

현재 처리 방식 결제 상태·PAID ledger·잔액·거래 이력을 한 DB 트랜잭션에서 반영합니다. 취소·환불 시 상태 전환과 재화 회수도 같은 경계에서 처리합니다.



Gemini API 티어 한도에 맞춰 사이트에서 요청량을 제한했습니다

모델별 요청량을 외부 API 호출 전에 검사하고, 초과 요청은 재화 차감 전에 사이트 API 에서 종료합니다.

모든 사용자가 하나의 Gemini API 키를 공유합니다. 사이트에서 Gemini API 티어의 모델별 한도를 넘겨 호출하면 Gemini 가 429 를 반환하고, 재화를 먼저 차감한 상태라면 환불이나 복구 처리가 필요합니다. 그래서 Flash-Lite-Flash-Pro 마다 별도 limiter 를 두고, Go API 가 재화 차감과 외부 호출을 시작하기 전에 요청을 검사하도록 했습니다.

핵심 구조 · 실제 코드를 읽기 쉽게 축약

```
if !utils.GlobalModelRateLimiter.Allow(s.SSID, s.LLMType) {
    utils.WriteError(w, http.StatusTooManyRequests, "Rate limit exceeded")
    return // 재화 차감과 Gemini 호출 전 종료
}
limiter := getLimiter(modelType) // 모델별 전역 token bucket
```



레이트리미터 설계

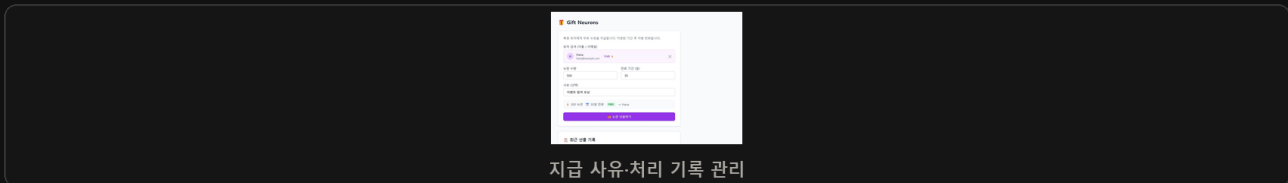
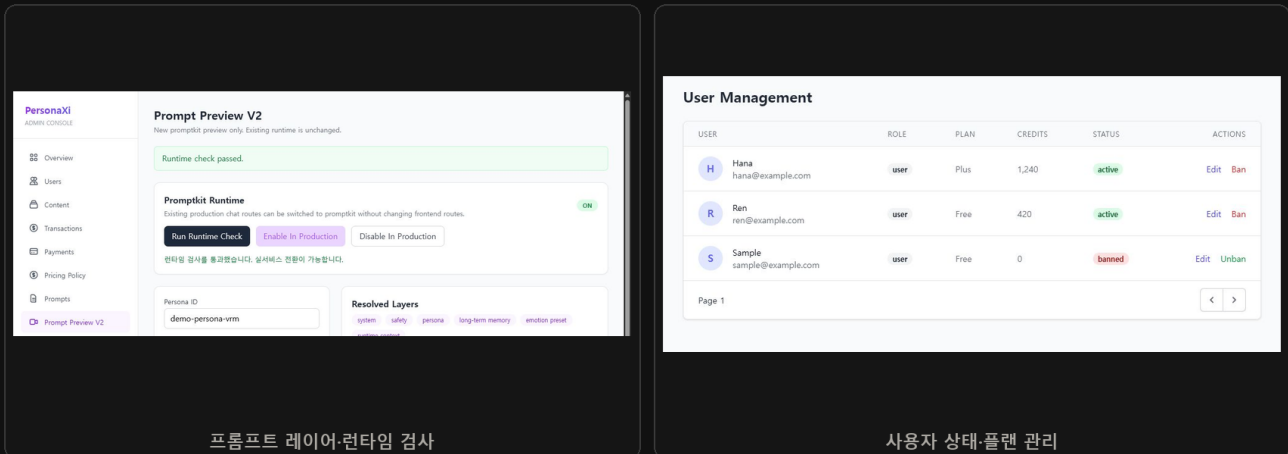
공유 단위	사용자별이 아니라 사이트가 공유하는 API 키의 모델별 요청량 제한
모델 분리	Flash-Lite-Flash-Pro 마다 별도 rate.Limiter 와 burst 값 사용
동시성	mutex 로 limiter map 의 최초 생성을 보호하고 생성한 limiter 를 재사용
적용 위치	모델 판별 후, 재화 차감과 외부 호출 전에 검사

구현 방식 Go 의 token bucket 으로 지속 요청량과 순간 burst 를 함께 제어했습니다. 일반 API 의 IP 기반 제한은 사용자 남용을 막는 용도로 두고, Gemini limiter 는 사이트 전체가 사용하는 API 티어의 한도를 넘지 않도록 별도로 관리했습니다.

관리자 페이지를 만들어 운영 상태를 관리했습니다

프롬프트 검사, 사용자 상태 확인, 재화 지급을 로컬 관리자 화면에 모았습니다.

프롬프트를 바꿀 때마다 코드나 DB 를 직접 수정하면 실수 범위가 커집니다. 조합 결과와 적용 레이어를 미리 확인하고, 런타임 검사를 통과한 설정만 서비스에 반영하도록 관리자 화면을 만들었습니다.



프롬프트	PromptKit 조합 미리보기, 모드별 결과 확인, 런타임 검사와 적용 상태 전환
사용자·보상	역할·플랜·상태·재화 조회, 지급 사유와 처리 기록 관리
접근 통제	화면은 로컬에서 실행하고 API 는 Supabase JWT-admin 역할·rate limit 으로 보호

배포·운영 환경

Frontend GitHub Pages - 로그인 이후 채팅 화면은 CSR 로 운영해 프론트엔드 호스팅 비용을 없앴습니다. 공개 페이지의 검색 노출이 중요해지면 해당 경로에만 SSR 이나 사전 렌더링을 적용할 수 있습니다.

Compute / Deploy Oracle Cloud + Coolify - API·PostgreSQL·Redis 와 배포·HTTPS·환경 변수·로그를 한곳에서 관리했습니다.

Assets / Model Supabase Storage + Hugging Face - 이미지 변환과 CPU 감정 분석을 분리해 전송량과 서버 비용을 줄였습니다.

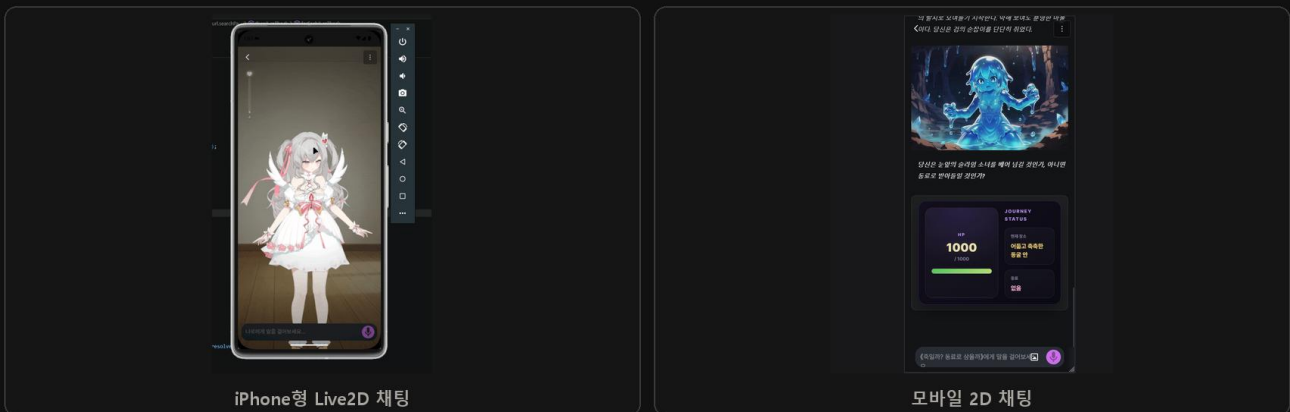
운영 중 반복되는 프롬프트 검증과 사용자 재화 지급은 관리자 기능으로 옮겼습니다.

입력창 위치와 이미지 로딩 문제를 브라우저에서 확인해 고쳤습니다

움직이는 영역과 전송되는 이미지 크기를 각각 줄였습니다.

1. iPhone Safari 입력창

VRM·Live2D 채팅은 전체 화면 canvas 위에 입력창이 고정됩니다. iPhone Safari 와 PWA 에서는 키보드가 열릴 때 화면 전체가 밀리고 입력창 위치가 흔들렸습니다. Visual Viewport 의 높이와 focus 상태를 관찰하고 입력 wrapper 만 이동하도록 수정했습니다.



보정 방식 $innerHeight - visualViewport.height - offsetTop$ 으로 키보드 영역을 계산하고, 전체 canvas 가 아닌 입력 wrapper 만 translateY 했습니다. Visual Viewport 를 지원하지 않는 환경에는 별도 보정을 적용하지 않습니다.

2. 썸네일 원본 이미지 로딩

캐릭터 허브의 첫 로딩이 오래 걸려 브라우저 Network 도구를 확인했습니다. 카드에 보이는 이미지는 작았지만 각 썸네일이 원본 해상도로 내려오고 있었습니다.

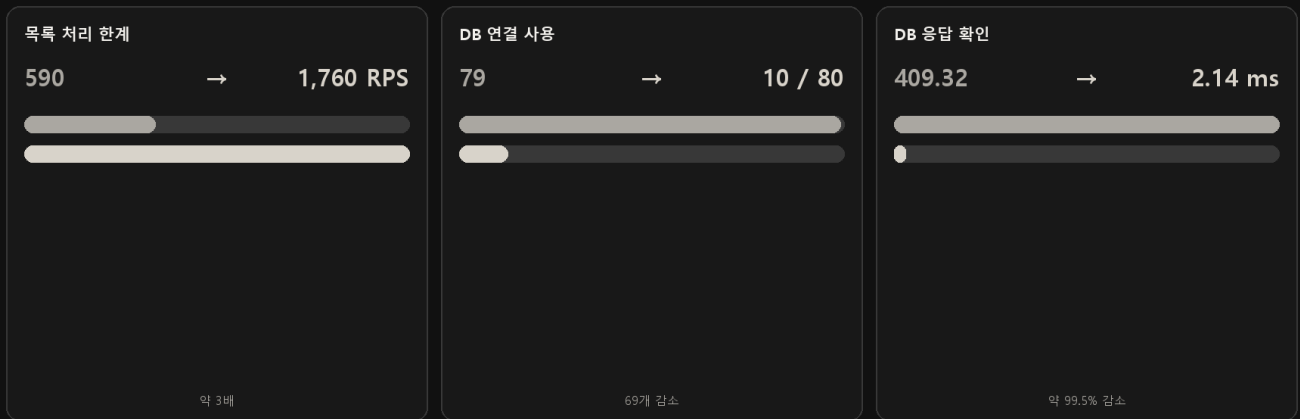


Supabase Storage URL 을 Image Transformation 경로로 바꾸는 공통 함수를 만들었습니다. 캐릭터 카드는 width 480-quality 72, 채팅 목록과 프로필 아이콘은 96×96 등 실제 표시 크기에 맞춰 요청하고, 상세 보기와 변환 실패 시에는 원본 URL 을 유지합니다.

k6 툴을 활용하여 웹사이트를 테스트해 본 과정

캐릭터 조회와 실제 채팅 흐름을 나누어 테스트했습니다.

캐릭터 목록 조회가 느려지고, 동시 채팅에서는 DB 연결이 빠르게 소진됐습니다. 캐릭터 조회와 2D 채팅을 별도 k6 시나리오로 만들어 캐시 효과와 저장 과정의 병목을 나눠 확인했습니다.



테스트와 수정

조회	같은 SQL 과 응답을 기준으로 Redis 적용 전후를 비교했습니다. p99 는 596.33ms 에서 239.04ms 로 줄었습니다.
채팅	캐릭터 선택·대화·대기 시간을 섞어 실제 사용 흐름과 비슷하게 구성했습니다. 수정 전에는 DB 연결 80 개 중 79 개를 사용했습니다.
저장	메시지와 세션을 한 번에 처리하고 재화 정산 기록을 함께 저장하도록 수정했습니다. 같은 조건에서 DB 연결 사용량은 79/80 에서 10/80 으로 줄었고, 응답 확인 시간은 409.32ms 에서 2.14ms 로 감소했습니다.

조회보다 채팅 저장 과정의 DB 연결 사용량이 더 큰 문제였습니다. 메시지와 세션 저장을 묶고 재화 정산 기록을 함께 처리하도록 바꾼 뒤 연결 사용량과 응답 확인 시간이 함께 줄었습니다.

디저트 판매자의 요구에 맞춰 주문과 운영 화면을 만들었습니다

실제 판매 과정에서 필요한 기능을 반영하며 수개월간 운영했습니다.

교내에서 디저트를 판매하던 지인의 요청으로 만들었습니다. 자체 도메인과 Ubuntu 서버에서 수개월간 운영했고, 예상보다 많은 사용자가 실제 주문에 이용했습니다.

고객 주문

메뉴·옵션·수령 시간 선택

판매자 관리

주문·재고·판매 일정 관리

운영 알림

Sheets 기록·Discord 알림

주문 정보·수령 시간 입력

주문 접수·결제 안내

주문·판매 처리 흐름

문제	메신저 주문은 메뉴·옵션·수령 시간 확인과 주문 정리에 반복 작업이 많음
고객 흐름	재고 확인 → 메뉴·옵션 → 연락처·수령 시간 → 주문 확인
판매자 흐름	관리자 인증 후 주문·메뉴·가격·재고·판매 일정·계좌 정보 관리
저장 / 알림	SQLite 트랜잭션과 mutex 처리 후 Google Sheets 기록·Discord 신규 주문 알림

운영 방식 필요 이상으로 복잡한 인프라보다 판매자가 바로 확인하고 수정할 수 있는 흐름을 우선했습니다. SQLite 트랜잭션으로 주문을 저장하고 Sheets와 Discord를 연결해 별도 교육 없이 사용할 수 있게 했습니다.

판매 중 추가한 기능 처음부터 기능 목록을 정해 만든 것이 아니라 실제 판매 중 나온 요청에 맞춰 메뉴·옵션·수령 시간·판매 일정·재고·계좌 정보와 주문 확인 UI를 계속 보완했습니다.

DEVELOPMENT EXPERIENCE

백엔드·서비스 운영을 중심으로 문제를 해결합니다

웹 서비스를 직접 만들고 운영하면서 사용한 기술과 프로젝트 링크를 정리했습니다.

주요 기술 경험

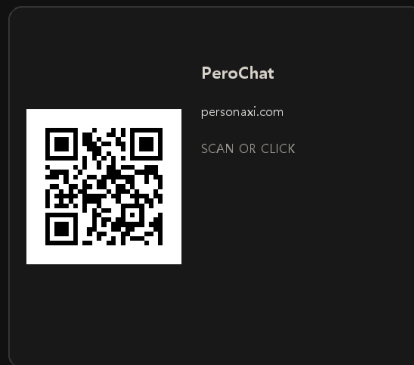
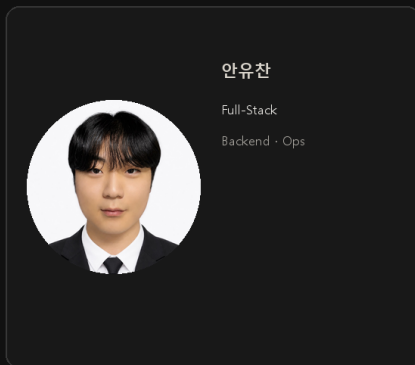
Backend / Data	Go · PostgreSQL · Redis · 결제 트랜잭션 · SSE / WebSocket · k6
Frontend	SvelteKit · TypeScript · CSR / PWA · Live2D / VRM · iPhone Safari 대응
Infrastructure	Oracle Cloud · Coolify · GitHub Pages · Supabase Auth / Storage
Game / Network	Unreal C++ · Unity C# · UDP ACK · NAT 홉핑 · 입력 예측 / 롤백

대표 링크

Service [PeroChat](#) · [personaxi.com](#)

Repository [PeroChat Frontend](#) · [Portfolio Viewer](#)

Game [Nirvana 플레이 영상](#)



AI 도구 활용

초안·번역·반복 작업에는 AI 에이전트를 활용했습니다. 아키텍처 결정, 핵심 통합, 오류 재현과 검증은 직접 수행하며 결과를 설명하고 수정할 수 있는 상태로 유지했습니다.

안유찬 · Full-Stack Developer · Backend & Service Operations

ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com